Project Design Phase
Problem – Solution Fit Template

Date	03 October 2026
Team ID	SWTID-2026-5297
Project Name	Script-Controlled ACL - Restrict Record Based on Field value
Maximum Marks	2 Marks

Problem – Solution Fit Template:

The Problem-Solution Fit Canvas maps the security gap in default ServiceNow record access against our custom script-controlled Access Control List (ACL) solution on the `u_institution_details` table.

Purpose:

- ❑ Solve complex data isolation and security access problems in a way that fits enterprise administrative workflows.
- ❑ Succeed faster by enforcing record-level security dynamically within the ServiceNow platform .
- ❑ Sharpen governance by mapping custom user roles (bb1–bb4) to script-based branch conditions (Branch = EEE) .
- ❑ Increase trust and compliance by preventing unauthorized data access or record modification across departments.

Template:

1. CUSTOMER SEGMENT(S) — CS

- **ServiceNow Admins:** Managing enterprise access controls.
- **Department Leads:** EEE department staff needing exclusive access to `u_institution_details`.
- **Audit & Compliance Officers:** Verifying data isolation.

6. CUSTOMER CONSTRAINTS — CC

- Strict platform performance requirements (no heavy query overhead).
- Standard out-of-the-box (OOTB) ACLs cannot perform dynamic field-matching across custom roles without custom scripting.

5. AVAILABLE SOLUTIONS — AS

- Static Role-Based Access Control (RBAC) without conditional filters.
- Separate database tables per department (leads to data fragmentation).
- Client-side UI Policies (insecure, easily bypassed via API).

2. JOBS-TO-BE-DONE — JBTD

- Restrict read/write/delete access on `u_institution_details`.
- Ensure users with roles bb1–bb4 only view records where `Branch = EEE`.
- Maintain impersonation logs for security audits.

9. PROBLEM ROOT CAUSE — RC

- Default ServiceNow ACLs grant access based on role presence alone without evaluating row-level column values dynamically unless configured with server-side JavaScript scripts.

7. BEHAVIOR — BH

- Users attempt to search or modify institution records via Workspace or REST APIs.
- Admins frequently use "Impersonate User" to verify permission boundaries.

3. TRIGGERS — TR

- Unauthorized access alerts during internal audit.
- Creation of new EEE department records requiring restricted visibility.

10. YOUR SOLUTION — SL

- **Script-Controlled Read/Write ACL:** Evaluates `gs.hasRole()` (bb1–bb4) AND `current.u_branch == 'EEE'` server-side before serving records.

8. CHANNELS OF BEHAVIOR — CH

- **Online:** ServiceNow Portal, Workspace UI, REST APIs.
- **Offline/Admin:** System Logs, Impersonation Audit trails, Security Dashboards.

4. EMOTIONS BEFORE / AFTER IMPLEMENTATION — EM

Before: Anxious about data leaks, uncertain if non-EEE users can view confidential institutional details via API or list view.

After: Confident, compliant, in complete control with automated server-side row-level security enforcement.